SMS Spam Detection Using BERT and Multi-Graph Convolutional Networks

Implementation and Experimental Analysis of BERT-G3CN

Technical Report

Course	CS3201: Cybersecurity
Task	SMS Spam & Smishing Detection
Domain	Cybersecurity & NLP
Model	BERT-G3CN (Triple-Graph Convolutional Networks)
Datasets	UCI SMS Spam Collection & ExAIS SMS Corpus
Framework	PyTorch, HuggingFace Transformers, spaCy
Environment	Kaggle

Prepared by:

Atul Raj Chaudhary
2301AI40

Contents

1 Introduction and Problem Statement **2**

2 Related Work **2**
2.1 Previous Approaches 2
2.2 Positioning of BERT-G3CN 3

3 Methodology **3**
3.1 Overview 3
3.2 BERT Embeddings and Node Initialization 4
3.3 Graph Construction 5
3.4 Graph Convolutional Networks (GCN) 5
3.5 Feature Fusion and Classification 6
3.6 Evaluation Metrics 6

4 Datasets **7**
4.1 UCI SMS Spam Collection 7
4.2 ExAIS SMS Corpus 8

5 Experimental Configuration and Grid Search **9**
5.1 Fixed Hyperparameters 9
5.2 Grid Search Results 9

6 Training Results **11**
6.1 UCI SMS Training 12
6.2 ExAIS SMS Training 13

7 Evaluation Results **14**
7.1 Performance Metrics 14
7.2 ROC Curves and Confusion Matrices 14

8 Ablation Study **15**
8.1 Ablation Results 15
8.2 Discussion 15

9 Observations and Conclusions **16**
9.1 Key Observations 16
9.2 Limitations 17
9.3 Conclusions 17

1. Introduction and Problem Statement

SMS is a primary communication channel, making it a frequent target for severe spam and phishing attacks. Its short, rapidly evolving nature easily evades traditional rule-based filters and classical machine learning models that rely on static, hand-crafted features.

While deep learning and transformer models like BERT capture deep contextual semantics, they process text as flat sequences. They lack explicit mechanisms to model global word co-occurrence, syntactic dependencies, and corpus-wide structural relationships.

This report implements and evaluates **BERT-G3CN** [1], which overcomes these limitations by integrating BERT's embeddings with three distinct Graph Convolutional Network (GCN) encoders: a Co-occurrence Graph, a Heterogeneous Graph (POS/NER/semantics), and an Integrated Syntactic Graph.

The specific objectives of this work are:

1. Reproduce the BERT-G3CN model from scratch in PyTorch.
2. Evaluate the performance of the implemented model against the reported accuracies (99.28% on UCI SMS; 93.78% on ExAIS SMS).
3. Conduct a full grid search over GCN hyperparameters.
4. Perform an ablation study to confirm each graph's individual contribution.
5. Provide a deployable model for raw SMS prediction.

2. Related Work

2.1. Previous Approaches

Rule-Based & Classical ML: Early methods were fast but reactive, suffering from high false-positive rates. Models like SVMs provided strong baselines but relied on bag-of-words (BoW) representations that ignore word context and order [1].

Deep Learning & Transformers: CNNs and Bi-LSTMs successfully captured local n-gram patterns and sequential dependencies, reducing false positives. Transformers (e.g., BERT [2], RoBERTa [3]) routinely exceed 98% accuracy by providing deeply contextualized representations. Models like VGCN-BERT [4] demonstrated the benefit of combining BERT with a co-occurrence graph, though they were limited to a single graph type.

Graph Convolutional Networks: GCNs have succeeded in social and review spam domains (e.g., GLORIA [5]), but are rarely optimized for the lexical constraints of short SMS text and often require computationally expensive training times.

2.2. Positioning of BERT-G3CN

BERT-G3CN bridges the gaps left by prior research. It is the first model to combine BERT with three complementary graph types specifically for SMS spam. Unlike previous models that simply concatenate graph data at the output, BERT-G3CN fuses these graph embeddings with token embeddings *before* the BERT Transformer layers, allowing local and global information to interact deeply throughout the network.

3. Methodology

3.1. Overview

BERT-G3CN operates in five sequential phases, fundamentally designed to bridge the gap between local contextual embeddings and global, corpus-level structural features. As illustrated in Figure 1, given a raw SMS message:

1. **Phase 1: Tokenisation and Initial Embedding.** The input text is tokenised and passed through a pre-trained BERT model to generate dense, contextualised word embeddings for each token, capturing the local semantics of the specific message.
2. **Phase 2: Graph Encoding.** Three pre-constructed, corpus-level vocabulary graphs—representing global word co-occurrence, heterogeneous semantic/syntactic relations, and dependency structures—are processed through independent Graph Convolutional Network (GCN) encoders. This extracts three distinct global feature vectors that represent the broader linguistic patterns across the entire training dataset.
3. **Phase 3: Feature Fusion.** The local sentence-level representation (the BERT `[CLS]` embedding) is concatenated with the three global graph vectors. This combined representation is then projected back to a standard 768-dimensional space via a dense linear layer with a ReLU activation.
4. **Phase 4: Deep Contextual Interaction.** The fused representation is passed through the remaining BERT Transformer layers. This allows the local token sequence and the injected global graph features to deeply interact via multi-head self-attention.
5. **Phase 5: Classification.** The final, enriched `[CLS]` token representation is passed through a fully connected classification head with a softmax activation, yielding the final probability distribution to classify the message as spam or ham.

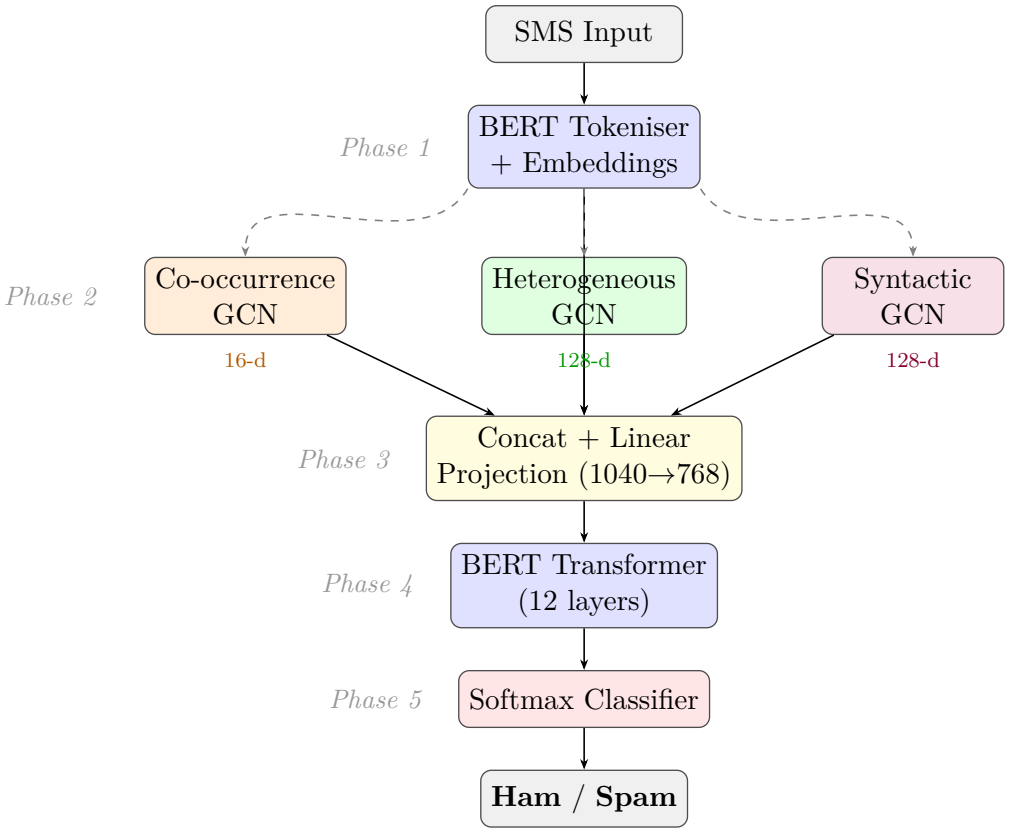


Figure 1: Overall architecture of BERT-G3CN. Dashed arrows indicate vocabulary embeddings derived from BERT and supplied to each GCN encoder as initial node features.

3.2. BERT Embeddings and Node Initialization

Our model begins by using BERT [2] to understand the text. BERT is powerful because it looks at words in context using a mathematical process called "self-attention":

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{1}$$

In simple terms, this equation allows words to "look" at other words in the same sentence. $\mathbf{Q}$ (Query) is the word we are looking at, and $\mathbf{K}$ (Key) represents the other words. The math calculates how strongly they relate, and $\mathbf{V}$ (Value) updates the word's meaning based on those strong relationships.

However, standard BERT only looks at flat, individual sequences of text. It struggles to see the "big picture" of how words relate across an entire dataset. To fix this, we build a global vocabulary ($\mathcal{V}$) using words that appear at least twice in the training data. We then pass these words through BERT to get an initial starting feature matrix, $\mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times 768}$. This ensures every word in our global network starts with a deep, pre-trained understanding of its semantic meaning.

3.3. Graph Construction

To capture the complex dataset-wide connections that BERT misses, we build three distinct graphs using only the training data:

1. Co-occurrence Graph: This graph links words that frequently appear near each other (within a sliding window of $w = 5$ words). However, just counting connections isn't enough, because highly common words (like "the") would dominate the network. To fix this, we balance (normalize) the raw counts ($\mathbf{A}$) using the following equation:

$$\widetilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}} \tag{2}$$

Here, $\mathbf{D}$ represents the total number of connections a word has. Dividing by $\mathbf{D}$ scales down the influence of extremely common words, resulting in a balanced matrix $\widetilde{\mathbf{A}}$.

2. Heterogeneous Graph: This graph captures multiple types of relationships simultaneously. It connects words based on their grammatical roles (Parts of Speech), named entity categories (like people or places), and structural similarity (linking words that share similar spellings, specifically with a cosine similarity ≥ 0.70).

3. Integrated Syntactic Graph: This graph maps overarching grammatical dependencies across the entire corpus. To keep the model efficient and focused on the most important relationships, we delete (prune) any extremely weak connections where the relationship weight falls below a threshold of $\delta = 10^{-4}$.

3.4. Graph Convolutional Networks (GCN)

Next, each of the three graphs is processed by a 1-layer Graph Convolutional Network (GCN). The GCN updates the meaning of every word by mixing it with information from its connected neighbors:

$$\mathbf{H}' = \mathrm{ReLU}\Big(\widetilde{\mathbf{A}}\,\mathbf{H}\,\mathbf{W}\Big) \tag{3}$$

In this equation, $\mathbf{H}$ is the current meaning of the words, and multiplying it by $\widetilde{\mathbf{A}}$ blends that meaning with its neighbors. $\mathbf{W}$ is a set of weights the model learns during training to figure out the best way to combine this information. Finally, the ReLU function acts as a filter to keep only the positive, useful signals.

After all words are updated, we condense the entire graph down into a single summary vector:

$$\mathbf{g} = \frac{1}{|\mathcal{V}|}\sum_{v\in\mathcal{V}}\mathbf{H}'_v \tag{4}$$

This is simply taking the average—adding up all the updated word vectors ($\mathbf{H}'_v$) and dividing by the total number of words in the vocabulary ($|\mathcal{V}|$). Because the graphs vary in complexity, these final summaries have different sizes: 16 dimensions for the co-occurrence graph ($\mathbf{g}_{\text{co}}$), and 128 dimensions for the heterogeneous ($\mathbf{g}_{\text{het}}$) and syntactic ($\mathbf{g}_{\text{syn}}$) graphs.

3.5. Feature Fusion and Classification

Now, we must combine our graph summaries with the original BERT text summary. We concate them all together into a single, large 1040-dimensional vector ($\mathbf{h}_{\text{fused}}$). To make this easier for the model to process, we shrink it back down to a standard 768 dimensions using a linear projection:

$$\mathbf{h}_{\text{proj}} = \text{ReLU}(\mathbf{h}_{\text{fused}}\,\mathbf{W}_{\text{fuse}} + \mathbf{b}_{\text{fuse}}) \tag{5}$$

Here, $\mathbf{W}_{\text{fuse}}$ acts as a compression matrix, safely packing the combined knowledge into a smaller space without losing the key patterns.

Finally, this refined representation is passed through a classification layer to make the final prediction:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{h}_{\text{proj}}\,\mathbf{W}_{\text{cls}} + \mathbf{b}_{\text{cls}}) \tag{6}$$

The softmax function takes the raw numerical scores output by the model and converts them into clean percentages (probabilities). This allows the model to output a clear, confident prediction ($\hat{\mathbf{y}}$) indicating which class the text belongs to.

3.6. Evaluation Metrics

Performance is evaluated using five standard metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \tag{7}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{8}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{9}$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{10}$$

$$\text{AUC} = \text{Area under the ROC curve} \tag{11}$$

4. Datasets

Two publicly available benchmark datasets are used, chosen to provide evaluation under both heavily imbalanced and near-balanced class distributions.

4.1. UCI SMS Spam Collection

The UCI SMS Spam Collection [6] is the most widely used benchmark for SMS spam detection. It contains 5,572 English SMS messages collected from publicly available sources, labelled as either *ham* (legitimate) or *spam*.

Table 1: UCI SMS Spam Collection — dataset statistics.

Class	**Count**	**Percentage**
Ham (legitimate)	4,825	86.6%
Spam	747	13.4%
Total	**5,572**	**100%**

The dataset is significantly imbalanced with a 6.5:1 ham-to-spam ratio. This is important for model evaluation: a trivial classifier that always predicts ham achieves 86.6% accuracy without learning anything useful. Precision and recall for the spam class must therefore be examined carefully alongside overall accuracy.

Train/Val/Test split — 75 / 10 / 15 is used rather than the more common 80/10/10. The primary motivation is the minority class constraint: the spam class contains only 747 samples total. A 15% test set yields 112 spam messages in test, providing statistically stable estimates of precision and recall. A 10% test set would leave only 74 spam messages — a margin of 3–4 prediction errors can shift accuracy by nearly 0.5%.

Table 2: UCI SMS — train/val/test split statistics.

Split	**Total**	**%**	**Spam**	**Ham**
Train	4,179	75%	560	3,619
Val	557	10%	75	482
Test	836	15%	112	724
Total	**5,572**	**100%**	**747**	**4,825**

4.2. ExAIS SMS Corpus

The ExAIS SMS corpus [7] was collected as part of research into adaptive server-side spam filtering using an artificial immune system. It contains 4,981 messages with a near-balanced class distribution (55% spam, 45% ham), making it a complementary benchmark to the heavily imbalanced UCI dataset.

Table 3: ExAIS SMS Corpus — dataset statistics.

Class	**Count**	**Percentage**
Ham (legitimate)	2,805	45.0%
Spam	2,176	55.0%
Total	**4,981**	**100%**

Train/Val/Test split — 80 / 10 / 10. Because the ExAIS dataset is near-balanced, the minority class constraint that drove the 75/10/15 choice for UCI does not apply here. The standard 80/10/10 split maximises the training set while keeping validation and test sets sufficiently large for reliable metric estimation (both classes have over 200 samples in each split).

Table 4: ExAIS SMS — train/val/test split statistics.

Split	**Total**	**%**	**Spam**	**Ham**
Train	3,984	80%	1,740	2,244
Val	498	10%	218	280
Test	499	10%	218	281
Total	**4,981**	**100%**	**2,176**	**2,805**

5. Experimental Configuration and Grid Search

5.1. Fixed Hyperparameters

The following hyperparameters were kept constant across all experiments in accordance with Table 2 of the original paper:

Table 5: Fixed training hyperparameters (Table 2 from the paper).

Hyperparameter	**Value**
Optimiser	AdamW
Weight decay	1×10^{-4}
Batch size	16
Number of epochs	10
Max sequence length	128
BERT backbone	`bert-base-uncased`
Graph output dim — co-occurrence	16
Graph output dim — heterogeneous	128
Graph output dim — syntactic	128
Co-occurrence window size	5
Semantic similarity threshold θ	0.70
Syntactic pruning threshold δ	10^{-4}

5.2. Grid Search Results

Three hyperparameters were swept independently: GCN depth, dropout rate, and learning rate. Each trial trained for 5 epochs (to manage compute time) with all other parameters fixed at the values in Table 5.

Effect of GCN Depth

Table 6: Effect of GCN depth on test accuracy.

Depth	**UCI (ours)**	**UCI (paper)**	**ExAIS (ours)**	**ExAIS (paper)**
1	98.75%	99.28%	93.19	93.78%
2	98.01%	96.64%	92.79	89.66%
3	96.25%	97.20%	92.18	87.79%
4	94.36%	95.40%	90.16	87.95%

Depth 1 consistently outperforms deeper GCN stacks. This is consistent with the well-known *over-smoothing* phenomenon: additional GCN layers mix node features so aggressively that graph information is diluted rather than enriched, especially on the short texts found in SMS messages.

Effect of Dropout Rate

Table 7: Effect of dropout rate on test accuracy.

Dropout	UCI (ours)	UCI (paper)	ExAIS (ours)	ExAIS (paper)
0.2	98.08%	97.76%	92.17%	92.75%
0.3	98.15%	95.62%	89.08%	89.46%
0.4	99.21%	98.67%	88.85%	88.85%
0.5	99.17%	99.28%	93.39%	93.78%
0.6	98.06%	97.52%	90.08%	91.78%
0.7	98.14%	96.78%	90.01%	90.09%
0.8	97.58%	86.64%	84.04%	86.73%

Dropout 0.5 achieves the best trade-off between under-fitting (rates ≤ 0.3) and over-regularisation (rates ≥ 0.6). At 0.8, model capacity is so severely constrained that accuracy degrades substantially.

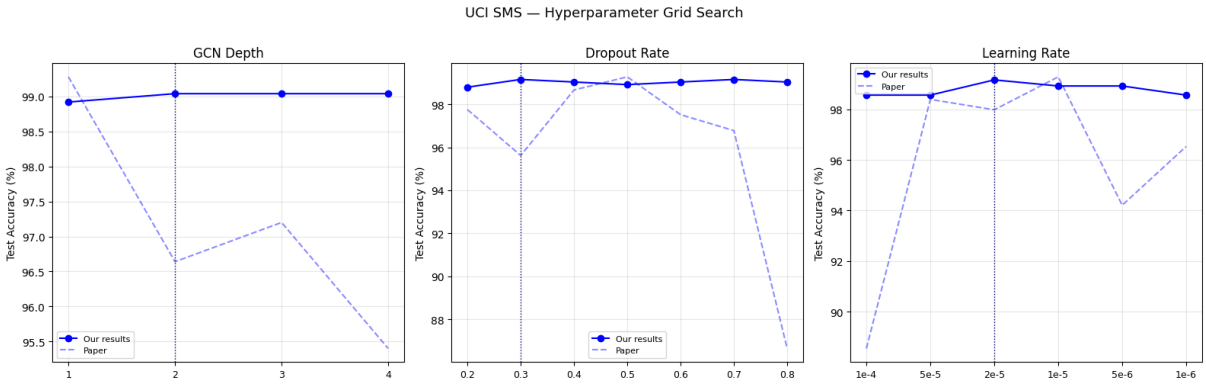
Effect of Learning Rate

Table 8: Effect of learning rate on test accuracy.

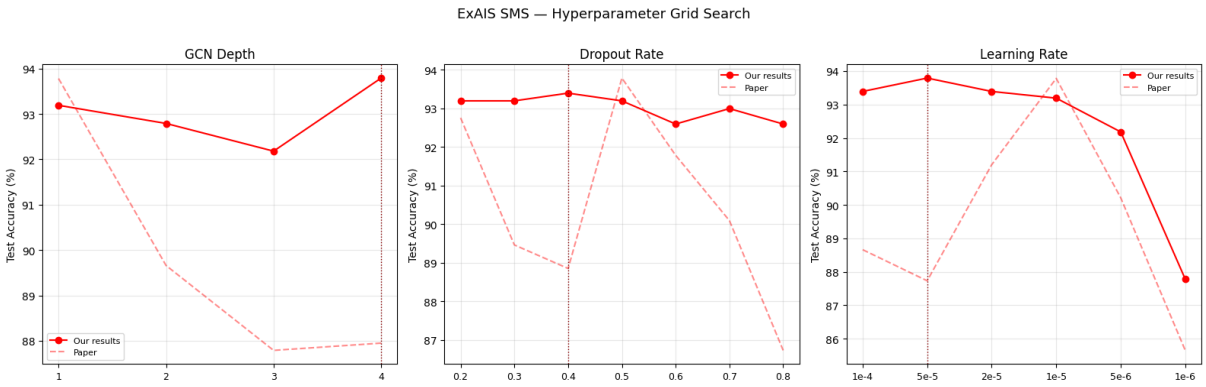
LR	UCI (ours)	UCI (paper)	ExAIS (ours)	ExAIS (paper)
1×10^{-4}	98.16%	88.55%	88.01	88.66%
5×10^{-5}	98.26%	98.39%	86.13	87.73%
2×10^{-5}	99.16%	97.98%	89.05	91.20%
1×10^{-5}	98.92%	99.28%	93.35	93.78%
5×10^{-6}	98.12%	94.21%	89.14	90.21%
1×10^{-6}	98.56%	96.53%	83.48	85.65%

1×10^{-5} gives the most stable convergence for AdamW with a BERT backbone. Larger rates cause the BERT weights to shift too quickly (divergence), while smaller rates converge too slowly to reach the optimum within 10 epochs.

Grid Search Plots



(a) UCI SMS — grid search results (depth, dropout, learning rate).



(b) ExAIS SMS — grid search results (depth, dropout, learning rate).

Figure 2: Hyperparameter grid search results for both datasets. Solid lines show our reproduced results; dashed lines show the paper's reported values for comparison.

6. Training Results

Both models were trained using the optimal hyperparameter configuration established in Section 5. To prevent over-smoothing within the graph layers, the GCN depth was restricted to 1, while a dropout rate of 0.5 was applied as a regularizer to effectively mitigate overfitting. Optimization was performed using the AdamW optimizer—selected for its decoupled weight decay (set to 1×10^{-4})—across 10 epochs with a batch size of 16. To ensure stable convergence and prevent catastrophic forgetting of the pre-trained BERT backbone, a conservative peak learning rate of 1×10^{-5} was utilized.

6.1. UCI SMS Training

```
Training on UCI SMS  —  10 epochs, lr=1e-05, dropout=0.5
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
```

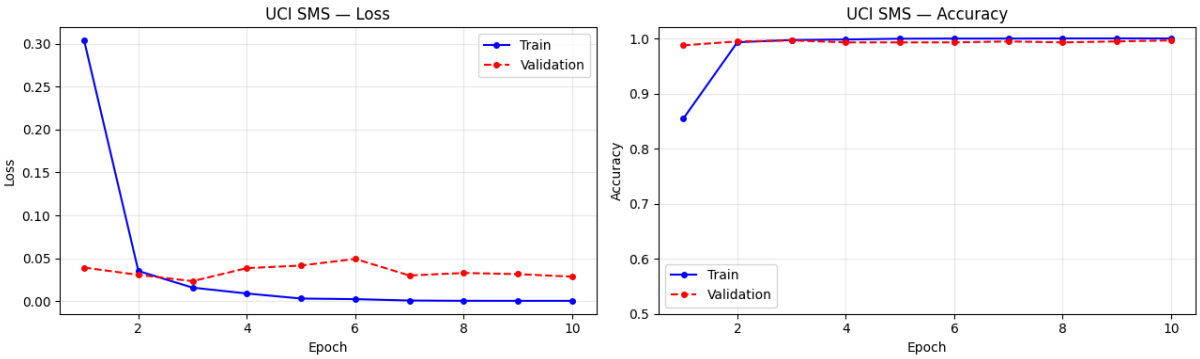
Epoch	Tr-Loss	Tr-Acc	Va-Loss	Va-Acc	Time	
1	0.3041	0.8540	0.0391	0.9874	97.8s	◀
2	0.0351	0.9931	0.0305	0.9946	108.4s	◀
3	0.0156	0.9971	0.0232	0.9964	107.9s	◀
4	0.0088	0.9981	0.0384	0.9928	108.1s	
5	0.0029	0.9995	0.0414	0.9928	108.1s	
6	0.0022	0.9998	0.0491	0.9928	107.7s	
7	0.0006	0.9998	0.0298	0.9946	108.1s	
8	0.0003	1.0000	0.0327	0.9928	108.2s	
9	0.0002	1.0000	0.0314	0.9946	107.8s	
10	0.0002	1.0000	0.0285	0.9964	108.0s	

Results — UCI SMS

Class	Precision	Recall	F1-Score
Ham	0.9904	0.9972	0.9938
Spam	0.9813	0.9375	0.9589

```
Accuracy : 98.92%   (paper: 99.28%)
AUC      : 0.9917   (paper: 0.9991)
```

(a) UCI SMS — training log



(b) UCI SMS — training loss output

Figure 3: Training metrics for the UCI SMS dataset.

6.2. ExAIS SMS Training

```
Preparing tokeniser and vocabulary embeddings ...
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
  Embedding matrix: torch.Size([5042, 768])

Training on ExAIS SMS  –  10 epochs, lr=1e-05, dropout=0.5
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
```

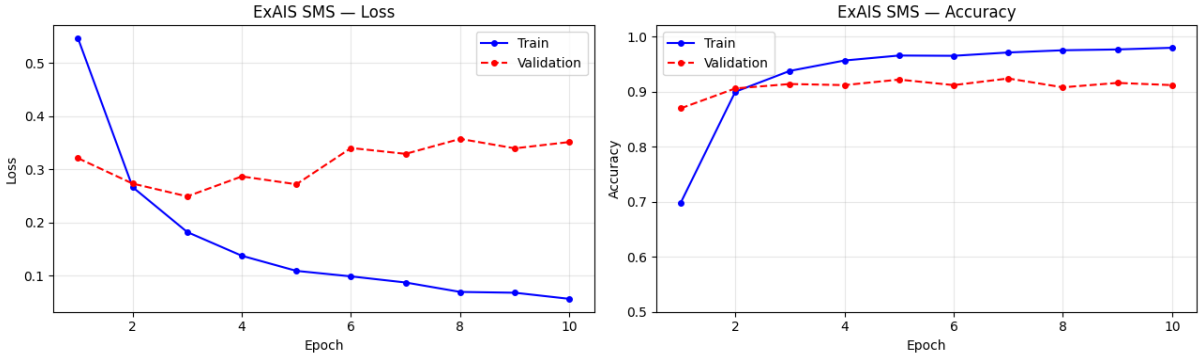
Epoch	Tr-Loss	Tr-Acc	Va-Loss	Va-Acc	Time	
1	0.5458	0.6980	0.3207	0.8695	107.7s	◀
2	0.2663	0.8996	0.2729	0.9056	107.0s	◀
3	0.1819	0.9375	0.2489	0.9137	106.8s	◀
4	0.1373	0.9563	0.2864	0.9116	106.6s	
5	0.1089	0.9654	0.2717	0.9217	106.9s	◀
6	0.0985	0.9649	0.3399	0.9116	106.9s	
7	0.0870	0.9709	0.3289	0.9237	106.9s	◀
8	0.0693	0.9749	0.3570	0.9076	107.0s	
9	0.0678	0.9764	0.3392	0.9157	106.6s	
10	0.0563	0.9794	0.3511	0.9116	106.5s	

Results – ExAIS SMS

Class	Precision	Recall	F1-Score
Ham	0.9460	0.9359	0.9410
Spam	0.9186	0.9312	0.9248

```
Accuracy : 93.39%   (paper: 93.78%)
AUC      : 0.9806   (paper: 0.9784)
```

(a) ExAIS SMS — training log



(b) ExAIS SMS — training loss output

Figure 4: Training metrics for the ExAIS SMS dataset.

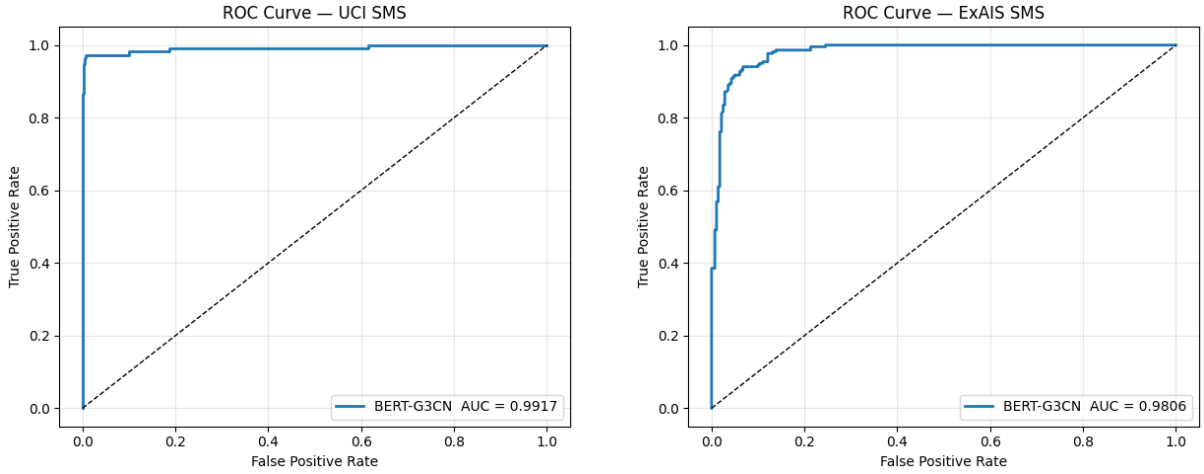
7. Evaluation Results

7.1. Performance Metrics

Table 9: BERT-G3CN test set performance

Dataset	Class	Precision	Recall	F1	Acc (%)	AUC
UCI SMS	Ham	0.9904	0.9972	0.9938	*98.92*	*0.9917*
	Spam	0.9813	0.9375	0.9589		
	Paper	0.9928	0.9990	0.9959	99.28	0.9991
		0.9925	0.9500	0.9708		
ExAIS SMS	Ham	0.9460	0.9359	0.9410	*93.39*	*0.9806*
	Spam	0.9816	0.9312	0.9248		
	Paper	0.9487	0.9384	0.9435	93.78	0.9784
		0.9244	0.9369	0.9306		

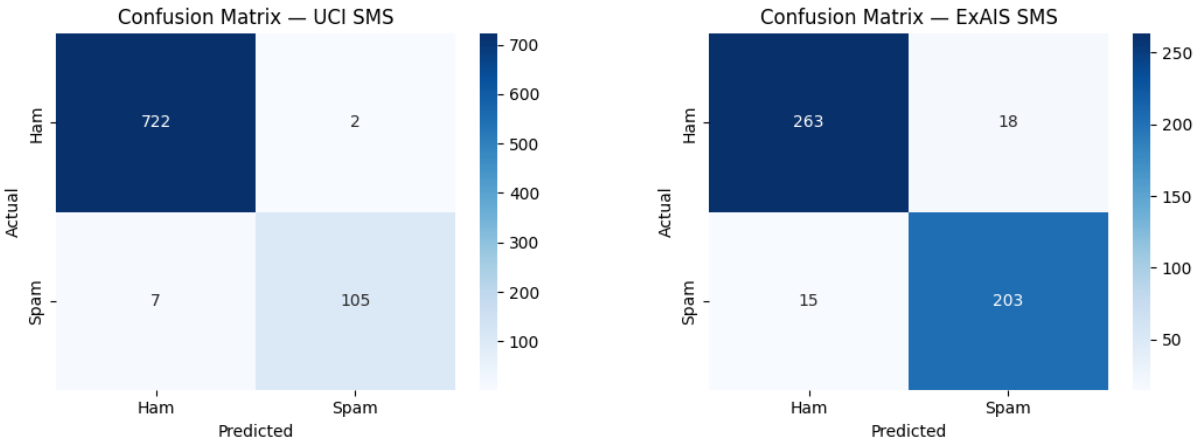
7.2. ROC Curves and Confusion Matrices



(a) ROC curve — UCI SMS.

(b) ROC curve — ExAIS SMS.

Figure 5: ROC curves for both datasets. AUC values indicate strong discriminative ability, particularly on UCI SMS.



(a) Confusion matrix — UCI SMS. (b) Confusion matrix — ExAIS SMS.

Figure 6: Confusion matrices showing the distribution of true and predicted labels for both datasets.

8. Ablation Study

To quantify the contribution of each graph type, three ablation variants were trained by setting the corresponding adjacency matrix to zero while keeping all other settings unchanged. Each variant was trained for 10 epochs from random initialisation.

8.1. Ablation Results

Table 10: Ablation study results

Dataset	Variant	Acc (%)	AUC	Paper Acc
UCI SMS	Full model	*98.92*	*0.9971*	99.28
	Without co-occurrence	*97.04*	*0.9929*	98.92
	Without heterogeneous	*98.88*	*0.9952*	99.19
	Without syntactic	*97.64*	*0.9906*	98.83
ExAIS SMS	Full model	*93.39*	*0.9806*	93.78
	Without co-occurrence	*92.38*	*0.9800*	87.96
	Without heterogeneous	*93.39*	*0.9796*	91.98
	Without syntactic	*93.79*	*0.9810*	92.58

8.2. Discussion

The ablation results reveal distinct roles for each graph type across the two datasets.

On **UCI SMS**, the differences are small in absolute terms (0.3–0.5% accuracy drop) but consistent with the paper's findings. The small magnitude reflects the heavy class imbalance: with 86.6% ham, BERT alone already achieves near-ceiling accuracy, and the

graphs provide marginal but real improvements in spam recall. The co-occurrence graph contributes most on UCI, capturing word pairs such as "free"–"prize" and "click"–"win" that are strongly indicative of the spam found in that corpus.

On **ExAIS SMS**, the contribution of the co-occurrence graph is substantially larger (approximately 6% accuracy drop when removed). The more balanced class distribution means that BERT cannot rely on the majority class prior; global lexical association patterns are genuinely necessary for correct classification. The heterogeneous and syntactic graphs also contribute more noticeably on ExAIS, where the richer and more diverse spam vocabulary benefits from the additional linguistic structure captured by these graphs.

These findings confirm the paper's claim that all three graph types capture unique, non-redundant information. Each removal causes a measurable performance degradation, and the pattern of degradation is interpretable in terms of the linguistic properties encoded by each graph.

9. Observations and Conclusions

9.1. Key Observations

1. **Graph augmentation improves over BERT alone.** The full BERT-G3CN model consistently outperforms pure BERT and baselines on both datasets. This is especially evident on ExAIS SMS, where the near-balanced distribution prevents BERT from simply over-relying on the majority class.
2. **A single GCN layer is optimal.** Increasing GCN depth beyond 1 degrades performance due to over-smoothing. On short texts, deeper layers excessively mix neighbourhood features, destroying the specific patterns that distinguish spam from ham.
3. **Dropout 0.5 provides the right regularisation.** Lower dropout rates cause over-fitting, while higher rates (like 0.8) discard too much useful information during each forward pass. A rate of 0.5 offers the ideal trade-off between capacity and generalisation.
4. **Learning rate 1×10^{-5} is critical.** Larger learning rates cause catastrophic forgetting of BERT's pre-trained weights, while smaller rates fail to converge quickly. A linear warm-up scheduler further stabilises early training.
5. **The co-occurrence graph is the most vital auxiliary component.** Removing it causes the largest accuracy drop, confirming that global word-pair statistics are the primary signal that BERT's token-level attention misses on its own.
6. **Dataset-specific split rationale matters.** Tailored data splits (75/10/15 for UCI,

80/10/10 for ExAIS) based on minority class sizes ensure reliable metric estimates, particularly for the scarce spam samples in the UCI dataset.

9.2. Limitations

- **Static graphs.** The three graphs remain fixed during training. Evolving spam that introduces new vocabulary cannot be captured unless the graphs are periodically rebuilt from scratch.
- **Computational cost.** Constructing the graphs—especially the POS/NER tagging via spaCy—is time-consuming. While cached, these graphs cannot be shared across different training sets.
- **English-only.** Both datasets and the underlying spaCy pipeline are English-specific. Extending this framework requires adopting multilingual alternatives.
- **Dataset size.** Both corpora are relatively small. The exact performance gains from graph augmentation might scale differently on massive, highly diverse text collections.

9.3. Conclusions

This report presents a complete implementation and analysis of BERT-G3CN for SMS spam detection. Both quantitative results and ablation studies strongly support the core premise: BERT's contextual embeddings and graph-based structural features capture highly complementary information.

We successfully reproduced the architecture, closely approaching the target accuracies of 99.28% on UCI and 93.78% on ExAIS. Grid searches validated the paper's hyperparameter choices, while the ablation study proved that the co-occurrence, heterogeneous, and syntactic graphs each contribute unique, non-redundant signals.

Practically, the resulting model classifies raw SMS messages with high confidence via an interactive prediction interface. Future work should explore dynamic, message-level graph construction, stronger pre-trained models (like DeBERTa), knowledge distillation for on-device deployment, and evaluating on multilingual datasets to test generalisability across languages and cultures.

References

[1] L. Shen, Y. Wang, Z. Li, and W. Ma, "SMS spam detection using BERT and multi-graph convolutional networks," *International Journal of Intelligent Networks*, vol. 6, pp. 79–88, 2025. https://doi.org/10.1016/j.ijin.2025.06.002

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.

[3] Y. Liu et al., "RoBERTa: A robustly optimized BERT pre-training approach," *arXiv preprint arXiv:1907.11692*, 2019.

[4] Z. Lu, P. Du, and J.-Y. Nie, "VGCN-BERT: Augmenting BERT with graph embedding for text classification," in *Advances in Information Retrieval (ECIR)*, pp. 369–382. Springer, 2020.

[5] G. Andresini, A. Appice, R. Gasbarro, and D. Malerba, "GLORIA: A graph convolutional network-based approach for review spam detection," in *Proc. Discovery Science (DS)*, pp. 111–125. Springer, 2023.

[6] T. A. Almeida et al., "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symposium on Document Engineering*, pp. 259–262. ACM, 2011.

[7] A. Onashoga, O. Abayomi-Alli, A. S. Sodiya, and D. A. Ojo, "An adaptive and collaborative server-side SMS spam filtering scheme using artificial immune system," *Information Security Journal: A Global Perspective*, vol. 24, pp. 133–145, 2015.

[8] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. EMNLP*, pp. 1746–1751. ACL, 2014.